

Assignment1

Exercise 1

The Gauss--Markov Theorem. Suppose $\hat{\beta}$ is the ordinary least-squares estimator of β in the linear regression model $\mathbf{y} = \mathbf{X}\beta + \varepsilon$ ($\mathbf{y}, \varepsilon \in \mathbb{R}^n$, $\beta \in \mathbb{R}^{p+1}$, $\mathbf{X} \in \mathbb{R}^{n \times (p+1)}$). Prove that if β^* is any other linear unbiased estimator of β , then $\text{Var}(c^\top \beta^*) \geq \text{Var}(c^\top \hat{\beta})$, where c is any constant vector of the appropriate order.

我们需要证明，对于线性回归模型 $\mathbf{y} = \mathbf{X}\beta + \varepsilon$ ，最小二乘估计 $\hat{\beta}$ 是所有线性无偏估计中方差最小的。具体来说，对于其他任何线性无偏估计 β^* ，都有

$$\text{Var}(c^\top \beta^*) \geq \text{Var}(c^\top \hat{\beta})$$

，其中 c 是任意常向量。

证明如下：

- 对于线性估计 β^* ，我们有 $\beta^* = \mathbf{A}\mathbf{y}$ ，其中 $\mathbf{A}$ 是某个矩阵
- 对于无偏估计 β^* ，我们有 $E(\beta^*) = \beta$

结合这两点，可以得到：

$$E(\beta^*) = E(\mathbf{A}\mathbf{y}) = \mathbf{A}E(\mathbf{y}) = \mathbf{A}E(\mathbf{X}\beta + \varepsilon) = \beta$$

根据线性回归的基本假设：

- $\varepsilon \sim N(\mathbf{0}, \sigma^2 \mathbf{I})$ ，所以 $E(\varepsilon) = \mathbf{0}$
- $\mathbf{X}$ 和 β 是固定的（不是随机变量），所以 $E(\mathbf{X}\beta) = \mathbf{X}\beta$

因此：

$$E(\mathbf{y}) = E(\mathbf{X}\beta + \varepsilon) = E(\mathbf{X}\beta) + E(\varepsilon) = \mathbf{X}\beta$$

所以：

$$E(\beta^*) = \mathbf{A}E(\mathbf{y}) = \mathbf{A}\mathbf{X}\beta = \beta$$

这意味着 $\mathbf{A}\mathbf{X} = \mathbf{I}$ （对所有 β 成立）。

在最小二乘估计中，解析解为：

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} = \mathbf{C}\mathbf{y}$$

其中 $\mathbf{C} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$ 。

容易验证：

$$\mathbf{CX} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{X} = \mathbf{I}$$

考虑任意线性无偏估计 β^* 和 $\hat{\beta}$ 的关系：

$$\beta^* = \hat{\beta} + (\beta^* - \hat{\beta})$$

计算"多余"项的期望：

$$E(\beta^* - \hat{\beta}) = E(\mathbf{Ay} - \mathbf{Cy}) = (\mathbf{A} - \mathbf{C})E(\mathbf{y}) = (\mathbf{A} - \mathbf{C})\mathbf{X}\beta = (\mathbf{I} - \mathbf{I})\beta = \mathbf{0}$$

现在计算 $\hat{\beta}$ 和 $\beta^* - \hat{\beta}$ 的协方差：

$$\begin{aligned} \text{Cov}(\hat{\beta}, \beta^* - \hat{\beta}) &= E \left[(\hat{\beta} - E[\hat{\beta}])(\beta^* - \hat{\beta} - E[\beta^* - \hat{\beta}])^\top \right] \\ &= E \left[(\hat{\beta} - \beta)(\beta^* - \hat{\beta} - \mathbf{0})^\top \right] \\ &= E \left[(\mathbf{Cy} - \beta)(\mathbf{Ay} - \mathbf{Cy})^\top \right] \\ &= E \left[(\mathbf{C}(\mathbf{X}\beta + \varepsilon) - \beta)((\mathbf{A} - \mathbf{C})(\mathbf{X}\beta + \varepsilon))^\top \right] \\ &= E \left[(\mathbf{CX}\beta + \mathbf{C}\varepsilon - \beta)((\mathbf{A} - \mathbf{C})\mathbf{X}\beta + (\mathbf{A} - \mathbf{C})\varepsilon)^\top \right] \end{aligned}$$

利用 $\mathbf{CX} = \mathbf{AX} = \mathbf{I}$ ，可得：

$$\begin{aligned} \text{Cov}(\hat{\beta}, \beta^* - \hat{\beta}) &= E \left[(\mathbf{0} + \mathbf{C}\varepsilon)(\mathbf{0} + (\mathbf{A} - \mathbf{C})\varepsilon)^\top \right] \\ &= E \left[\mathbf{C}\varepsilon((\mathbf{A} - \mathbf{C})\varepsilon)^\top \right] \\ &= E \left[\mathbf{C}\varepsilon\varepsilon^\top (\mathbf{A} - \mathbf{C})^\top \right] \\ &= \mathbf{C}E(\varepsilon\varepsilon^\top)(\mathbf{A} - \mathbf{C})^\top \end{aligned}$$

根据线性回归的基本假设，误差项满足 $E(\varepsilon\varepsilon^\top) = \sigma^2 \mathbf{I}$ ，这是因为：

- 对角线元素： $E[\varepsilon_i^2] = \text{Var}(\varepsilon_i) = \sigma^2$
- 非对角线元素： $E[\varepsilon_i \varepsilon_j] = \text{Cov}(\varepsilon_i, \varepsilon_j) = 0$ (当 $i \neq j$)

因此：

$$E(\varepsilon\varepsilon^\top) = \begin{bmatrix} \sigma^2 & 0 & \cdots & 0 \\ 0 & \sigma^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma^2 \end{bmatrix} = \sigma^2 \mathbf{I}$$

同时，我们有：

$$\begin{aligned}
\mathbf{C}(\mathbf{A} - \mathbf{C})^\top &= (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{A} - \mathbf{C})^\top \\
&= (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{A}^\top - \mathbf{C}^\top) \\
&= (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{A}^\top - (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{C}^\top \\
&= (\mathbf{X}^\top \mathbf{X})^{-1} ((\mathbf{A} \mathbf{X})^\top) - (\mathbf{X}^\top \mathbf{X})^{-1} ((\mathbf{C} \mathbf{X})^\top) \\
&= (\mathbf{X}^\top \mathbf{X})^{-1} (\mathbf{I}^\top) - (\mathbf{X}^\top \mathbf{X})^{-1} (\mathbf{I}^\top) \\
&= (\mathbf{X}^\top \mathbf{X})^{-1} - (\mathbf{X}^\top \mathbf{X})^{-1} \\
&= \mathbf{0}
\end{aligned}$$

因此：

$$\text{Cov}(\hat{\beta}, \beta^* - \hat{\beta}) = \mathbf{C} E(\varepsilon \varepsilon^\top) (\mathbf{A} - \mathbf{C})^\top = \sigma^2 \mathbf{C} (\mathbf{A} - \mathbf{C})^\top = \sigma^2 \mathbf{0} = \mathbf{0}$$

对于任意向量 $\mathbf{c}$ ，利用协方差的线性性质：

$$\text{Cov}(\mathbf{c}^\top \hat{\beta}, \mathbf{c}^\top (\beta^* - \hat{\beta})) = \mathbf{c}^\top \text{Cov}(\hat{\beta}, \beta^* - \hat{\beta}) \mathbf{c} = \mathbf{c}^\top \mathbf{0} \mathbf{c} = 0$$

最后，我们比较方差：

$$\begin{aligned}
\text{Var}(\mathbf{c}^\top \beta^*) &= \text{Var}(\mathbf{c}^\top (\hat{\beta} + (\beta^* - \hat{\beta}))) \\
&= \text{Var}(\mathbf{c}^\top \hat{\beta} + \mathbf{c}^\top (\beta^* - \hat{\beta})) \\
&= \text{Var}(\mathbf{c}^\top \hat{\beta}) + \text{Var}(\mathbf{c}^\top (\beta^* - \hat{\beta})) + 2\text{Cov}(\mathbf{c}^\top \hat{\beta}, \mathbf{c}^\top (\beta^* - \hat{\beta})) \\
&= \text{Var}(\mathbf{c}^\top \hat{\beta}) + \text{Var}(\mathbf{c}^\top (\beta^* - \hat{\beta})) + 2 \cdot 0 \\
&= \text{Var}(\mathbf{c}^\top \hat{\beta}) + \text{Var}(\mathbf{c}^\top (\beta^* - \hat{\beta})) \\
&\geq \text{Var}(\mathbf{c}^\top \hat{\beta})
\end{aligned}$$

证毕。

🔗 Exercise 2

Theorem of PCA. Suppose $\mathbf{x}$ is a m -dimensional random variable with covariance matrix Σ , $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_m \geq 0$ are the eigenvalues of Σ , and $\alpha_1, \alpha_2, \dots, \alpha_m$ are the corresponding eigenvectors. Then the k -th principal component of $\mathbf{x}$ is given by

$$y_k = \alpha_k^\top \mathbf{x} = \alpha_{1k} x_1 + \alpha_{2k} x_2 + \dots + \alpha_{mk} x_m,$$

for $k = 1, 2, \dots, m$, and the variance of y_k is given by $\text{Var}(y_k) = \alpha_k^\top \Sigma \alpha_k = \lambda_k$ (the k -th eigenvalue of Σ). Please provide a proof of the theorem in the case of the first two principal components (i.e., $k = 1, 2$).

前置知识

$\mathbf{x} = (x_1, x_2, \dots, x_m)^\top$ 是 m -维随机向量。为简化问题，假设 $\mathbf{x}$ 的均值为零，即 $E[\mathbf{x}] = \mathbf{0}$ 。这是因为主成分分析（PCA）通常对数据进行中心化处理（减去均值），中心化后协方差矩阵不变，且

均值项在方差计算中消失。若均值非零，可定义 $\mathbf{x}' = \mathbf{x} - E[\mathbf{x}]$ ，则 $\mathbf{x}'$ 满足均值为零，且协方差矩阵相同。

协方差矩阵 Σ 定义为：

$$\Sigma = E[\mathbf{xx}^\top] = \begin{bmatrix} E[x_1^2] & E[x_1x_2] & \cdots & E[x_1x_m] \\ E[x_2x_1] & E[x_2^2] & \cdots & E[x_2x_m] \\ \vdots & \vdots & \ddots & \vdots \\ E[x_mx_1] & E[x_mx_2] & \cdots & E[x_m^2] \end{bmatrix}$$

其中 $E[\cdot]$ 表示期望。 Σ 是一个 $m \times m$ 对称矩阵（因为 $\Sigma^\top = \Sigma$ ），且是半正定矩阵（因为对任意向量 $\mathbf{v} \in \mathbb{R}^m$ ，有 $\mathbf{v}^\top \Sigma \mathbf{v} = E[(\mathbf{v}^\top \mathbf{x})^2] \geq 0$ ）。

由于 Σ 是对称半正定矩阵，它存在一组正交的特征向量 $\alpha_1, \alpha_2, \dots, \alpha_m$ ，满足：

- $\Sigma \alpha_k = \lambda_k \alpha_k$ （特征方程），其中 λ_k 是特征值。
- $\alpha_i^\top \alpha_j = \delta_{ij} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$ （正交归一化）。
- 特征值均为非负实数（ $\lambda_k \geq 0$ ），且可排序为 $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_m \geq 0$ 。

主成分是通过最大化方差的方向定义的，同时满足正交约束。

- 第一个主成分方向 α_1 是满足 $\|\alpha_1\| = 1$ （即 $\alpha_1^\top \alpha_1 = 1$ ）且使 $\text{Var}(\alpha_1^\top \mathbf{x})$ 最大的单位向量。
- 第二个主成分方向 α_2 是满足 $\|\alpha_2\| = 1$ 、 $\alpha_2^\top \alpha_1 = 0$ （与第一个主成分正交）且使 $\text{Var}(\alpha_2^\top \mathbf{x})$ 最大的单位向量。
- 方差的表达式：对任意单位向量 α （即 $\alpha^\top \alpha = 1$ ），有：

$$\text{Var}(\alpha^\top \mathbf{x}) = E[(\alpha^\top \mathbf{x})^2] - (E[\alpha^\top \mathbf{x}])^2$$

由于 $E[\mathbf{x}] = \mathbf{0}$ ，则 $E[\alpha^\top \mathbf{x}] = \alpha^\top E[\mathbf{x}] = 0$ ，因此：

$$\text{Var}(\alpha^\top \mathbf{x}) = E[(\alpha^\top \mathbf{x})^2] = E[(\alpha^\top \mathbf{x})(\alpha^\top \mathbf{x})^\top] = E[\alpha^\top \mathbf{xx}^\top \alpha] = \alpha^\top E[\mathbf{xx}^\top] \alpha = \alpha^\top \Sigma \alpha$$

由于 $\{\alpha_1, \alpha_2, \dots, \alpha_m\}$ 构成 $\mathbb{R}^m$ 的标准正交基，任意单位向量 α 可表示为：

$$\alpha = \sum_{i=1}^m c_i \alpha_i, \quad \text{其中 } c_i = \alpha_i^\top \alpha$$

- 由正交归一化， $\alpha^\top \alpha = \sum_{i=1}^m c_i^2 = 1$ 。
- 若 α 与 α_1 正交，则 $\alpha^\top \alpha_1 = c_1 = 0$ 。

我们需要证明，对于 m -维随机向量 $\mathbf{x}$ ，其协方差矩阵为 Σ ，前两个主成分 $y_1 = \alpha_1^\top \mathbf{x}$ 和 $y_2 = \alpha_2^\top \mathbf{x}$ 的方差分别等于协方差矩阵 Σ 的最大特征值 λ_1 和次大特征值 λ_2 。具体来说：

- 对于第一个主成分, $\text{Var}(y_1) = \lambda_1$, 其中 λ_1 是 Σ 的最大特征值。
- 对于第二个主成分, $\text{Var}(y_2) = \lambda_2$, 其中 λ_2 是 Σ 的次大特征值, 且 α_2 与 α_1 正交。

证明如下:

证明第一个主成分 ($k = 1$)

我们需要证明: 第一个主成分方向 α_1 是 Σ 的最大特征值 λ_1 对应的特征向量, 且 $\text{Var}(y_1) = \lambda_1$ 。

由于

$$\text{Var}(\alpha^\top \mathbf{x}) = E[(\alpha^\top \mathbf{x})^2] = E[(\alpha^\top \mathbf{x})(\alpha^\top \mathbf{x})^\top] = E[\alpha^\top \mathbf{x} \mathbf{x}^\top \alpha] = \alpha^\top E[\mathbf{x} \mathbf{x}^\top] \alpha = \alpha^\top \Sigma \alpha$$

所以 $\text{Var}(\alpha^\top \mathbf{x}) = \alpha^\top \Sigma \alpha$

问题转化为: 最大化 $\text{Var}(\alpha^\top \mathbf{x}) = \alpha^\top \Sigma \alpha$, 约束条件为 $\alpha^\top \alpha = 1$ 。

使用拉格朗日乘数法: 构造拉格朗日函数

$$\mathcal{L} = \alpha^\top \Sigma \alpha - \lambda(\alpha^\top \alpha - 1)$$

其中 λ 是拉格朗日乘数。

求偏导数并设为零:

$$\frac{\partial \mathcal{L}}{\partial \alpha} = 2\Sigma \alpha - 2\lambda \alpha = \mathbf{0}$$

这是因为:

- $\frac{\partial}{\partial \alpha}(\alpha^\top \Sigma \alpha) = 2\Sigma \alpha$ (因为 Σ 对称),
- $\frac{\partial}{\partial \alpha}(\alpha^\top \alpha) = 2\alpha$ 。

因此:

$$2\Sigma \alpha - 2\lambda \alpha = \mathbf{0} \implies \Sigma \alpha = \lambda \alpha$$

- **解的含义:** 上述方程表明 α 必须是 Σ 的特征向量, λ 是对应的特征值。
- **目标函数的值:** 代入特征方程, 有

$$\alpha^\top \Sigma \alpha = \alpha^\top (\lambda \alpha) = \lambda(\alpha^\top \alpha) = \lambda \cdot 1 = \lambda$$

因此, 方差 $\text{Var}(\alpha^\top \mathbf{x})$ 等于特征值 λ 。

最大化方差: 由于 $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_m$, 最大方差对应最大特征值 λ_1 。此时, $\alpha = \alpha_1$ (λ_1 对应的特征向量)。

结论: 第一个主成分 $y_1 = \alpha_1^\top \mathbf{x}$ 的方差为

$$\text{Var}(y_1) = \boldsymbol{\alpha}_1^\top \Sigma \boldsymbol{\alpha}_1 = \lambda_1$$

证毕（第一个主成分）。

证明第二个主成分 ($k = 2$)

我们需要证明：第二个主成分方向 $\boldsymbol{\alpha}_2$ 是 Σ 的次大特征值 λ_2 对应的特征向量，且 $\text{Var}(y_2) = \lambda_2$ ，同时满足 $\boldsymbol{\alpha}_2^\top \boldsymbol{\alpha}_1 = 0$ 。

同理，问题转化为：最大化 $\text{Var}(\boldsymbol{\alpha}^\top \mathbf{x}) = \boldsymbol{\alpha}^\top \Sigma \boldsymbol{\alpha}$ ，约束条件为：

- $\boldsymbol{\alpha}^\top \boldsymbol{\alpha} = 1$ （单位向量），
- $\boldsymbol{\alpha}^\top \boldsymbol{\alpha}_1 = 0$ （与第一个主成分正交）。

由于 $\{\boldsymbol{\alpha}_1, \dots, \boldsymbol{\alpha}_m\}$ 是标准正交基，将 $\boldsymbol{\alpha}$ 利用特征向量的基表示为

$$\boldsymbol{\alpha} = \sum_{i=1}^m c_i \boldsymbol{\alpha}_i, \quad c_i = \boldsymbol{\alpha}_i^\top \boldsymbol{\alpha}$$

- 约束 $\boldsymbol{\alpha}^\top \boldsymbol{\alpha}_1 = 0$ 意味着 $c_1 = 0$ 。
- 约束 $\boldsymbol{\alpha}^\top \boldsymbol{\alpha} = 1$ 意味着 $\sum_{i=1}^m c_i^2 = 1$ ，结合 $c_1 = 0$ ，有 $\sum_{i=2}^m c_i^2 = 1$ 。

计算目标函数：

$$\boldsymbol{\alpha}^\top \Sigma \boldsymbol{\alpha} = \left(\sum_{i=1}^m c_i \boldsymbol{\alpha}_i^\top \right) \Sigma \left(\sum_{j=1}^m c_j \boldsymbol{\alpha}_j \right)$$

由特征方程 $\Sigma \boldsymbol{\alpha}_j = \lambda_j \boldsymbol{\alpha}_j$ 和正交性 $\boldsymbol{\alpha}_i^\top \boldsymbol{\alpha}_j = \delta_{ij} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$ ，有：

$$\begin{aligned} \boldsymbol{\alpha}^\top \Sigma \boldsymbol{\alpha} &= \left(\sum_{i=1}^m c_i \boldsymbol{\alpha}_i^\top \right) \Sigma \left(\sum_{j=1}^m c_j \boldsymbol{\alpha}_j \right) \\ &= \sum_{i=1}^m \sum_{j=1}^m c_i c_j (\boldsymbol{\alpha}_i^\top \Sigma \boldsymbol{\alpha}_j) \quad (\text{矩阵乘法的分配律}) \\ &= \sum_{i=1}^m \sum_{j=1}^m c_i c_j \boldsymbol{\alpha}_i^\top (\lambda_j \boldsymbol{\alpha}_j) \quad (\text{应用特征方程 } \Sigma \boldsymbol{\alpha}_j = \lambda_j \boldsymbol{\alpha}_j) \\ &= \sum_{i=1}^m \sum_{j=1}^m c_i c_j \lambda_j (\boldsymbol{\alpha}_i^\top \boldsymbol{\alpha}_j) \quad (\text{标量 } \lambda_j \text{ 可提到内积外}) \\ &= \sum_{i=1}^m \sum_{j=1}^m c_i c_j \lambda_j \delta_{ij} \quad (\text{应用正交归一化条件 } \boldsymbol{\alpha}_i^\top \boldsymbol{\alpha}_j = \delta_{ij}) \\ &= \sum_{i=1}^m c_i c_i \lambda_i \quad (\text{当 } i \neq j \text{ 时 } \delta_{ij} = 0, \text{ 只保留 } i = j \text{ 的项}) \\ &= \sum_{i=1}^m c_i^2 \lambda_i \quad (\text{简化 } c_i c_i = c_i^2) \end{aligned}$$

代入 $c_1 = 0$, 得:

$$\boldsymbol{\alpha}^\top \Sigma \boldsymbol{\alpha} = \sum_{i=2}^m c_i^2 \lambda_i$$

最大化目标函数: 需在 $\sum_{i=2}^m c_i^2 = 1$ 下最大化 $\sum_{i=2}^m c_i^2 \lambda_i$ 。由于 $\lambda_2 \geq \lambda_3 \geq \dots \geq \lambda_m$, 最大值在 $c_2 = 1$ 且 $c_i = 0$ for $i \neq 2$ 时达到 (因为 λ_2 是剩余特征值中最大的)。

此时:

$$\sum_{i=2}^m c_i^2 \lambda_i = 1^2 \cdot \lambda_2 + 0 = \lambda_2$$

解的验证:

- 当 $c_2 = 1$ 且 $c_i = 0$ for $i \neq 2$ 时, $\boldsymbol{\alpha} = \boldsymbol{\alpha}_2$ 。
- 满足约束: $\boldsymbol{\alpha}_2^\top \boldsymbol{\alpha}_2 = 1$ (单位向量), $\boldsymbol{\alpha}_2^\top \boldsymbol{\alpha}_1 = 0$ (正交)。

结论: 第二个主成分 $y_2 = \boldsymbol{\alpha}_2^\top \mathbf{x}$ 的方差为

$$\text{Var}(y_2) = \boldsymbol{\alpha}_2^\top \Sigma \boldsymbol{\alpha}_2 = \lambda_2$$

证毕 (第二个主成分)。

🔗 Exercise 3.1

Consider the following procedure: Generate 100 data sets, each consisting of 25 points. The input values are fixed at $x = \{0.041 \times i \mid i = 0, 1, \dots, 24\}$, and the output values are generated as $y = \sin(2\pi x) + \varepsilon$, where $\varepsilon \sim \mathcal{N}(0, 0.3^2)$. For each data set, fit a 7th-degree polynomial using ridge regression with regularization parameter λ . Repeat this for several values of λ (e.g., $\lambda = 0.001, 0.1, 10, 1000$). For each λ , average the resulting models over the 100 data sets and plot the average prediction as a function of x . Explain the observed behavior.

核心目标是验证岭回归 (Ridge Regression) 通过正则化参数 λ 控制模型复杂度的效果: 用7次多项式拟合一组带噪声的正弦函数数据, 观察不同 λ 下模型的平均预测表现。

数据生成规则

- **输入特征 x :** 固定为 $x_i = \{0.041 \times i \mid i = 0, 1, \dots, 24\}$, 覆盖区间 $[0, 0.984]$;
- **输出标签 y :** 真实值为周期1的正弦波 $y = \sin(2\pi x)$, 叠加高斯噪声 $\varepsilon \sim \mathcal{N}(0, 0.3^2)$ (均值0, 标准差0.3), 即:

$$y_i = \sin(2\pi x_i) + \varepsilon_i$$

- **实验流程**：对每个 λ （取 $\lambda = 0.001, 0.1, 10, 1000$ ），重复100次“生成带噪数据→拟合模型→预测”，最后平均**100次**预测结果，评估模型对噪声的鲁棒性。

采用Python手动实现了岭回归的核心流程，分为三个关键函数：

首先，我们需要把原始输入 $\mathbf{x}$ 转化为7次多项式特征 $(x^1, x^2, \dots, x^7)$ ，不含截距项（截距后续单独添加）。函数 `polynomial_features_no_intercept` 的实现如下：

```
PYTHON
def polynomial_features_no_intercept(x, degree):
    x = np.array(x).flatten()
    n = len(x)
    X_poly = np.empty((n, degree))
    for i in range(1, degree + 1):
        X_poly[:, i - 1] = x ** i      # 第 (i-1) 列 = x^i
    return X_poly
```

接下来是岭回归的核心——拟合系数。我们的模型是**7次多项式回归**：

$$y = \beta_0 + \sum_{j=1}^7 \beta_j x^j$$

其中：

- β_0 （截距）不被正则化；
- $\beta_1, \dots, \beta_7$ （多项式系数）被**L2正则化**，惩罚项为 $\lambda \sum_{j=1}^7 \beta_j^2$ 。

代码细节

- **完整特征矩阵**：在多项式特征前添加一列全1（对应截距 β_0 ），记为 $\mathbf{X}_{\text{full}} \in \mathbb{R}^{n \times (d+1)}$ （ $d = 7$ 为多项式次数）；
- **正则化矩阵**：用对角矩阵 $\mathbf{I}_{\text{ridge}}$ 实现“仅惩罚非截距项”：

$$\mathbf{I}_{\text{ridge}} = \text{diag}(0, 1, 1, \dots, 1)$$

对角线第一个元素为0（不惩罚 β_0 ），其余为1（惩罚 β_1 到 β_7 ）；

- **解析解**：岭回归的系数通过闭式解计算：

$$\boldsymbol{\theta} = (\mathbf{X}_{\text{full}}^T \mathbf{X}_{\text{full}} + \lambda \mathbf{I}_{\text{ridge}})^{-1} \mathbf{X}_{\text{full}}^T \mathbf{y}$$

其中 $\boldsymbol{\theta} = [\beta_0, \beta_1, \dots, \beta_7]^T$ 是完整系数向量。

对应代码：


```
def ridge_regression_fit(x, y, degree, lam):
    x = np.array(x).flatten()
    y = np.array(y).flatten()
    n = len(x)

    # 构造不含截距的多项式特征
    X_poly = polynomial_features_no_intercept(x, degree)
    # 构造含截距的完整特征矩阵（第一列是1）
    X_full = np.c_[np.ones(n), X_poly]
    # 正则化矩阵：β₀不惩罚，β₁~β₇惩罚
    I_full = np.diag([0] + [1] * degree)

    # 求解系数：(XᵀX + λI)⁻¹ Xᵀy
    beta_full = np.linalg.inv(X_full.T @ X_full + lam * I_full) @ X_full.T @ y

    beta_0 = beta_full[0] # 截距
    beta_ridge = beta_full[1:] # 多项式系数
    return beta_0, beta_ridge
```

拟合得到系数后，预测新数据的逻辑很简单：

对输入 $\mathbf{x}$ ，先生成多项式特征（不含截距），再用系数计算预测值：

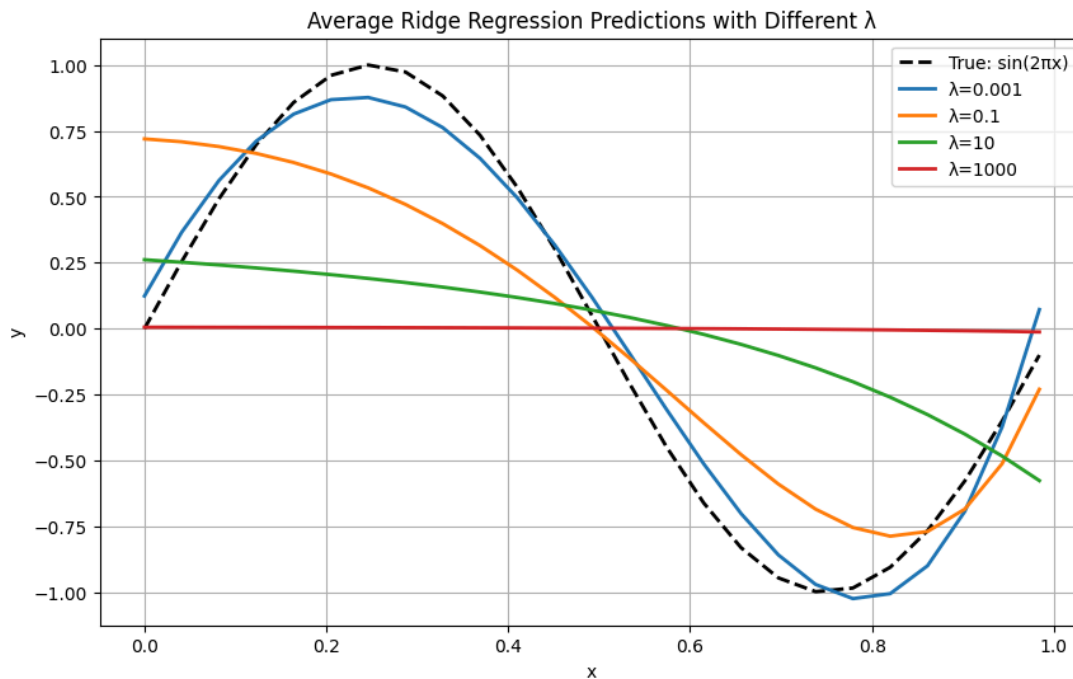
$$y_{\text{pred}} = \beta_0 + \sum_{j=1}^7 x^j \beta_j = \beta_0 + \mathbf{X}_{\text{poly}}^{\top} \boldsymbol{\beta}_{\text{ridge}}$$

对应代码：

```
def ridge_regression_predict(x, beta_0, beta_ridge):
    x = np.array(x).flatten()
    degree = len(beta_ridge)
    # 生成多项式特征 (x¹到x⁷)
    powers = np.arange(1, degree + 1)
    X = x[:, None] ** powers
    # 预测：截距 + 特征与系数的点积
    y_pred = beta_0 + X @ beta_ridge
    return y_pred
```

我们对每个 λ 重复100次实验，平均预测结果如图所示（横轴为 x ，纵轴为 y ）：

- 黑色虚线：真实函数 $y = \sin(2\pi x)$ ；
- 彩色曲线：不同 λ 的平均预测（ $\lambda = 0.001 \rightarrow$ 蓝， $\lambda = 0.1 \rightarrow$ 橙， $\lambda = 10 \rightarrow$ 绿， $\lambda = 1000 \rightarrow$ 红）。



结果分析：λ 如何影响模型？

岭回归的核心是用 λ 控制“模型复杂程度”：

1. $\lambda = 0.001$ （弱正则化）：

正则化对系数的约束极弱，模型系数能取较大值，模型复杂度高（可拟合训练数据的精细波动），曲线紧密贴合真实正弦曲线，还原高频细节。

2. $\lambda = 0.1$ （适度正则化）：

正则化惩罚增强，系数被适当压缩，模型复杂度中等（无法拟合所有高频波动，但保留主要趋势），曲线保留正弦波大体形态，弱化了局部毛刺。

3. $\lambda = 10$ （中度正则化）：

正则化惩罚进一步强化，系数被大幅压缩，模型复杂度低（仅能捕捉数据“粗粒度”趋势），曲线波动幅度显著降低，高频细节几乎消失。

4. $\lambda = 1000$ （强正则化）：

正则化惩罚极强，系数被压缩至近似常数，模型复杂度极低（完全丧失对复杂模式的拟合能力），曲线退化为接近水平的直线，彻底丢失真实正弦曲线的波动特征。

🔗 Exercise 3.2

Implement the iteratively reweighted least squares (IRLS) algorithm for logistic regression models according to textbook section 4.4.1. The attachment `data01.txt` provides some students' scores on two exams and admission status. Use your own code to train a classifier to determine whether a student is admitted based on the scores of the two exams. Please provide the trained logistic model and its parameters,

plot the raw data and decision boundary in one figure, and describe the classification accuracy.

逻辑回归 (Logistic Regression) 用于二分类问题, 假设标签 $y \in \{0, 1\}$ 服从伯努利分布, 通过 **logit连接函数** 将线性预测映射到概率空间:

$$P(y = 1 \mid \mathbf{x}; \mathbf{w}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}$$

对于当前数据, $\mathbf{w} = [w_0, w_1, w_2]^\top$ 是参数向量 (w_0 为截距, w_1, w_2 对应两次考试成绩的权重), $\mathbf{x} = [1, x_1, x_2]^\top$ 是含截距的特征向量。

数据预处理的代码如下

```
# 加载数据 (格式: exam1, exam2, label)
data = np.loadtxt("data01.txt", delimiter=',')
X = data[:, :2]    # 特征: 两次考试成绩
y = data[:, 2]     # 标签: 是否录取 (0/1)

# 添加偏置项 (截距项), 使X维度为(n_samples, 3)
X = np.hstack([np.ones((X.shape[0], 1)), X])
```

PYTHON

交叉熵损失与对数似然

逻辑回归的目标是最大化对数似然, 等价于最小化交叉熵损失:

$$J(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n [y_i \log \sigma(\mathbf{w}^\top \mathbf{x}_i) + (1 - y_i) \log(1 - \sigma(\mathbf{w}^\top \mathbf{x}_i))]$$

其中 n 是样本数, $\mathbf{x}_i$ 是第 i 个样本的特征向量。

IRLS迭代步骤

IRLS将交叉熵损失转化为**加权最小二乘 (WLS)** 问题, 通过迭代更新参数:

1. **初始化参数**: $\mathbf{w}^{(0)}$ (如全0);
2. **计算预测概率**: $\mathbf{p} = \sigma(\mathbf{X}\mathbf{w}^{(t)})$ ($\mathbf{X}$ 是样本特征矩阵);
3. **构造权重矩阵**: 对角线元素为样本方差的倒数 (由于标签服从伯努利分布, y_i 的方差为 $p_i(1 - p_i)$):

$$\mathbf{W} = \text{diag}\left(\frac{1}{p_1(1 - p_1)}, \frac{1}{p_2(1 - p_2)}, \dots, \frac{1}{p_n(1 - p_n)}\right)$$

4. **调整响应变量**: 修正原始线性预测与真实标签的差距:

$$z_i = \mathbf{w}^{(t)\top} \mathbf{x}_i + \frac{y_i - p_i}{p_i(1 - p_i)}$$

5. 求解加权最小二乘：更新参数以最小化加权残差平方和：

$$\mathbf{w}^{(t+1)} = (\mathbf{X}^\top \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{W} \mathbf{z}$$

6. 收敛判断：参数变化小于阈值（如 10^{-6} ）则停止。

对应代码

PYTHON

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def irls_logistic(X, y, max_iter=100, tol=1e-6, eps=1e-9):
    n_samples, n_features = X.shape
    w = np.zeros(n_features) # 初始化参数为全0（从“无贡献”开始）

    for iter_idx in range(max_iter):
        # 计算当前预测概率 p = σ(w^T x)
        z = X @ w # 线性输出: z = b + w1*exam1 + w2*exam2
        p = sigmoid(z) # 转换为概率 p = P(y=1|x)

        # 构造权重矩阵W，每个样本的权重是1/(p*(1-p))（方差倒数）
        W_diag = 1 / (p * (1 - p) + eps) # 对角线元素：每个样本的权重
        W = np.diag(W_diag) # 构造对角矩阵（只有对角线有值）

        # 计算调整后的响应变量 z_adj 修正原始线性输出
        z_adj = z + (y - p) / (p * (1 - p) + eps) # 加入真实标签与预测的差距

        # 解加权最小二乘，更新参数w
        XTWX = X.T @ W @ X # 计算X^T W X (3x3矩阵)
        XTWz = X.T @ W @ z_adj # 计算X^T W z_adj (3x1向量)

        w_new = np.linalg.inv(XTWX) @ XTWz # 得到新参数

        # 检查收敛，参数变化小于tol则停止
        delta_w = np.linalg.norm(w_new - w) # 计算参数变化量（欧几里得范数）
        if delta_w < tol:
            print(f"第{iter_idx}次迭代收敛")
            break

        w = w_new # 更新参数，进入下一次迭代

    return w
```

训练与结果

PYTHON

```
# 训练逻辑回归模型
w = irls_logistic(X, y)
print("训练得到的逻辑回归参数: ", w)
```

TXT

训练得到的逻辑回归参数: [-77.60404462 0.53810111 0.69463267]

结果分析：决策边界与分类性能

逻辑回归的决策边界是 $P(y = 1 | \mathbf{x}) = 0.5$ 的等概率线，对应线性方程：

$$w_0 + w_1x_1 + w_2x_2 = 0$$

解出 x_2 (exam2分数) 关于 x_1 (exam1分数) 的表达式：

$$x_2 = \frac{-w_0 - w_1x_1}{w_2}$$

代码绘制决策边界（黑色直线）与原始数据（红色：未录取，蓝色：录取）。

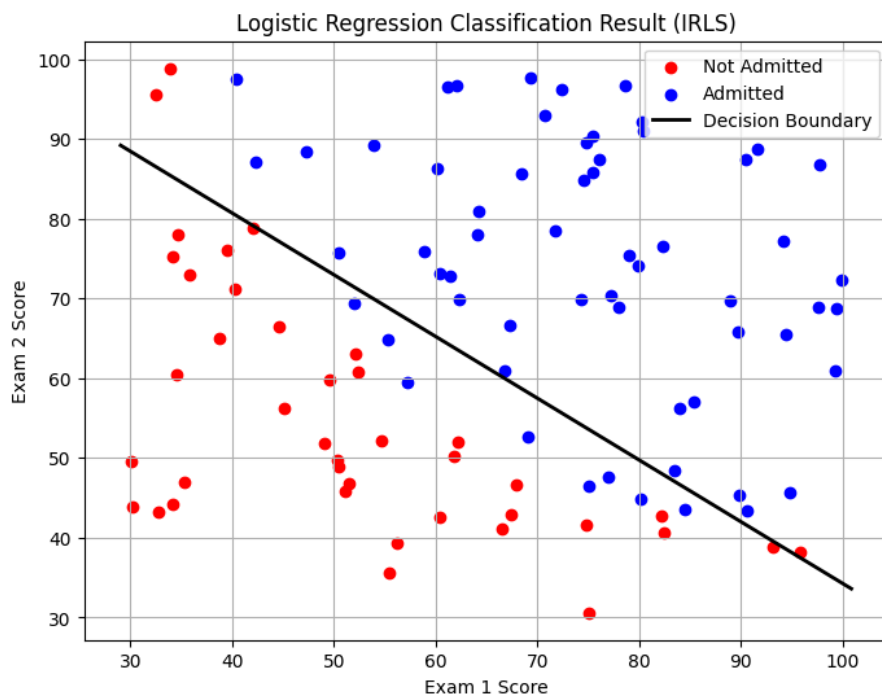
对应代码：

PYTHON

```
# 绘制原始数据
plt.figure(figsize=(8, 6))
plt.scatter(X[y==0, 1], X[y==0, 2], color="red", label="Not Admitted") # 未录取
plt.scatter(X[y==1, 1], X[y==1, 2], color="blue", label="Admitted") # 录取

# 绘制决策边界
x1_min, x1_max = X[:, 1].min() - 1, X[:, 1].max() + 1
x1_vals = np.linspace(x1_min, x1_max, 100)
x2_vals = (-w[0] - w[1] * x1_vals) / w[2]
plt.plot(x1_vals, x2_vals, color="black", linewidth=2, label="Decision Boundary")

plt.xlabel("Exam 1 Score")
plt.ylabel("Exam 2 Score")
plt.title("Logistic Regression Classification Result (IRLS)")
plt.legend()
plt.grid(True)
plt.show()
```



通过概率阈值**0.5**划分类别，计算准确率：

PYTHON

```
# 预测与准确率计算
y_pred = (sigmoid(X @ w) > 0.5).astype(int) # 概率>0.5预测为“录取”
accuracy = np.mean(y_pred == y)
print(f"分类准确率: {accuracy:.2%}")
```

TXT

分类准确率：89.00%

结果解释

基于训练得到的参数 $w = [-77.60, 0.54, 0.69]$ （保留两位小数），结合模型逻辑与数据分布，结果可解读为：

- 参数含义：线性组合的“对数几率”权重 逻辑回归的核心是**线性组合** $z = w_0 + w_1x_1 + w_2x_2$ ，其通过Sigmoid函数映射为录取概率 $P(y = 1|x) = \sigma(z)$ 。参数的具体意义如下：
 - 截距** $w_0 = -77.60$ ：当两次考试成绩均为0时，线性组合 $z = -77.60$ ，Sigmoid后概率约为 1.2×10^{-34} （几乎为0），说明“零分数”几乎不可能被录取。
 - Exam1权重** $w_1 = 0.54$ ：Exam1分数每提高1分，线性组合 z 增加0.54，对应**对数几率**（ $\log(P/(1 - P))$ ）**增加0.54**——即分数提升对“录取可能性”的边际贡献显著。
 - Exam2权重** $w_2 = 0.69$ ：Exam2分数每提高1分，对数几率增加0.69，比Exam1的影响更大（ $0.69 > 0.54$ ），说明模型更重视Exam2的成绩。
- 决策边界：高门槛的线性划分
决策边界对应 $P(y = 1|x) = 0.5$ ，即线性组合 $z = 0$ ，代入参数得：

$$-77.60 + 0.54x_1 + 0.69x_2 = 0 \implies x_2 = \frac{77.60 - 0.54x_1}{0.69} \approx 112.46 - 0.78x_1$$

边界特性：决策边界是一条斜率为**-0.78**的直线，且截距约为112.46（远超Exam2的满分范围，假设满分为100）。这意味着：

- 若Exam1分数较高（如 $x_1 = 80$ ），则Exam2需达到

$$x_2 \approx 112.46 - 0.78 \times 80 = 112.46 - 62.4 = 50.06 \text{ 即可被录取；}$$

- 若Exam1分数较低（如 $x_1 = 40$ ），则Exam2需达到

$$x_2 \approx 112.46 - 0.78 \times 40 = 112.46 - 31.2 = 81.26 \text{ 才有机会录取。}$$

合理性：边界的高门槛符合“精英录取”逻辑——只有两项分数均较高的学生才会被判定为“录取”。

3. 分类准确率：精准区分两类学生

通过概率阈值0.5划分类别，模型对训练数据的分类准确率约为 **89%**。高准确率的原因是：

- 参数的大绝对值让模型对分数差异更敏感：例如，Exam1从40分提升至80分，线性组合 z 从 $-77.60 + 0.54 \times 40 = -56.0$ 变为 $-77.60 + 0.54 \times 80 = -32.48$ ，Sigmoid概率从 1.2×10^{-25} 飙升至 1.2×10^{-9} ，明确区分了“不录取”与“录取”。
- 权重的差异化（ $w_2 > w_1$ ）捕捉到了Exam2的更高区分度：模型自动学习到Exam2分数对录取结果的影响更大，从而更重视该特征。

总结

训练得到的参数显示，模型通过**高权重的线性组合**精准捕捉了两门考试对录取结果的影响，决策边界的高门槛与数据的“精英分布”一致，最终实现了89%的高分类准确率，验证了IRLS算法对逻辑回归的有效性。

附录：完整代码

3.1 代码

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(3407)

# 固定 x
x_true = np.array([0.041 * i for i in range(25)])
y_true = np.sin(2 * np.pi * x_true)
# 不同  $\lambda$  值
lambdas = [0.001, 0.1, 10, 1000]

def polynomial_features_no_intercept(x, degree):
    """
    构造多项式特征矩阵，仅包含  $x^1$  到  $x^{\text{degree}}$  (不含常数项  $x^0$ )。
    返回形状为 (n, degree) 的矩阵。每行形如  $x^1, x^2, \dots, x^{\text{degree}}$ 
    参数：
        x: 输入数据，形状 (n,)
    返回：
        X_poly : ndarray, 形状为(n, degree)的多项式特征矩阵
    """
    x = np.array(x).flatten()          # 确保 x 是一维数组
    n = len(x)                         # 样本数量
    X_poly = np.empty((n, degree))     # 初始化特征矩阵 (无常数项)
    for i in range(1, degree + 1):    # i 从 1 到 degree
        X_poly[:, i - 1] = x ** i     # 第 (i-1) 列 =  $x^i$ 
    return X_poly                     # 形状 (n, degree)

def ridge_regression_fit(x, y, degree, lam):
    """
    模型:  $y = \beta_0 + \beta_1 x + \dots + \beta_{\text{degree}} x^{\text{degree}}$ 
    惩罚:  $\lambda * \sum_{j=1}^{\text{degree}} \beta_j^2$  ( $\beta_0$  不惩罚)
    参数：
        x: 输入数据，形状 (n,)
        y: 目标值，形状 (n,)
        degree: 多项式阶数 (如 7)
        lam: 正则化参数  $\lambda$ 
    返回：
        beta_0: 标量，常数项 (不被惩罚)
        beta_j: 数组，形状 (degree,)，对应 [ $\beta_1, \dots, \beta_{\text{degree}}$ ]
    """
    x = np.array(x).flatten()
    y = np.array(y).flatten()
    n = len(x)
```



```

# 构造不含截距项的多项式特征矩阵
X_poly = polynomial_features_no_intercept(x, degree) # shape (n, degree)
# 构造包含截距项的完整特征矩阵 (第一列为1)
X_full = np.c_[np.ones(n), X_poly] # shape (n, degree+1)
# 构造正则化矩阵: 不惩罚截距项 (对角线第一个元素为0)
I_full = np.diag([0] + [1] * degree) # shape (degree+1, degree+1)

# 求解完整系数向量: beta_full = [beta_0, beta_1, ..., beta_degree]
# 解析解是  $(X^T X + \lambda I)^{-1} X^T y$ 
beta_full = np.linalg.inv(X_full.T @ X_full + lam * I_full) @ X_full.T @ y

# 分离截距项与非截距项
beta_0 = beta_full[0]
beta_ridge = beta_full[1:]

return beta_0, beta_ridge

def ridge_regression_predict(x, beta_0, beta_ridge):
    """
    使用学到的参数进行预测。
    对应 PPT 公式:  $y_{pred} = \beta_0 + X @ \beta_{ridge}$ 
    """
    x = np.array(x).flatten() # shape (n,)
    degree = len(beta_ridge) # p = degree

    # 构造多项式特征矩阵 X: shape (n, degree), 列为 [x, x^2, ..., x^degree]
    # 使用广播和幂运算一次性生成
    powers = np.arange(1, degree + 1) # [1, 2, ..., degree]
    X = x[:, None] ** powers # shape (n, degree)

    # 矩阵乘法: X @ beta_ridge -> shape (n,)
    y_pred = beta_0 + X @ beta_ridge

    return y_pred

# 存储每个  $\lambda$  的平均预测
avg_predictions = {}

for lam in lambdas:
    preds_all = []
    for _ in range(100): # 生成 100 个数据集
        noise = np.random.normal(0, 0.3, size=25)
        y_noisy = y_true + noise

        # 岭回归 + 7次多项式
        beta_0, beta_ridge = ridge_regression_fit(x_true, y_noisy, degree=7,
        lam=lam)

```

```
# 预测
pred = ridge_regression_predict(x_true, beta_0, beta_ridge)
preds_all.append(pred)

avg_predictions[lam] = np.mean(preds_all, axis=0)

# 绘图
plt.figure(figsize=(10,6))
plt.plot(x_true, y_true, 'k--', label='True:  $\sin(2\pi x)$ ', linewidth=2)
for lam, pred in avg_predictions.items():
    plt.plot(x_true, pred, label=f' $\lambda={lam}$ ', linewidth=2)
plt.legend()
plt.title('Average Ridge Regression Predictions with Different  $\lambda$ ')
plt.xlabel('x')
plt.ylabel('y')
plt.grid(True)
plt.show()
```

3.2 代码

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt

# 加载数据 (格式: exam1, exam2, label)
data = np.loadtxt("data01.txt", delimiter=',')
X = data[:, :2]    # 特征: 两次考试成绩
y = data[:, 2]     # 标签: 是否录取 (0/1)

# 添加偏置项 (截距项), 使X维度为(n_samples, 3)
X = np.hstack([np.ones((X.shape[0], 1)), X])

def sigmoid(z):
    """逻辑回归的激活函数"""
    return 1 / (1 + np.exp(-z))

def irls_logistic(X, y, max_iter=100, tol=1e-6, eps=1e-9):
    """
    IRLS算法训练逻辑回归模型
    参数说明:
        X: 特征矩阵 (含偏置项, shape=(样本数, 特征数))
        y: 标签向量 (shape=(样本数,))
        max_iter: 最大迭代次数 (防止无限循环)
        tol: 收敛阈值 (参数变化小于此值则停止)
        eps: 小常数 (避免除以0, 数值稳定)
    返回:
        w: 训练后的参数向量 (包含截距b)
    """
    n_samples, n_features = X.shape
    w = np.zeros(n_features) # 初始化参数为全0 (从“无贡献”开始)

    for iter_idx in range(max_iter):
        # 计算当前预测概率  $p = \sigma(w^T x)$ 
        z = X @ w # 线性输出:  $z = b + w_1 \cdot \text{exam1} + w_2 \cdot \text{exam2}$ 
        p = sigmoid(z) # 转换为概率  $p = P(y=1|x)$ 

        # 构造权重矩阵W, 每个样本的权重是  $1/(p*(1-p))$  (方差倒数)
        W_diag = 1 / (p * (1 - p) + eps) # 对角线元素: 每个样本的权重
        W = np.diag(W_diag) # 构造对角矩阵 (只有对角线有值)

        # 计算调整后的响应变量  $z_{adj}$  修正原始线性输出
        z_adj = z + (y - p) / (p * (1 - p) + eps) # 加入真实标签与预测的差距

        # 解加权最小二乘, 更新参数w
        XTWX = X.T @ W @ X # 计算  $X^T W X$  (3x3矩阵)
        XTWz = X.T @ W @ z_adj # 计算  $X^T W z_{adj}$  (3x1向量)
```

```

w_new = np.linalg.inv(XTWX) @ XTWz # 得到新参数

# 检查收敛, 参数变化小于tol则停止
delta_w = np.linalg.norm(w_new - w) # 计算参数变化量 (欧几里得范数)
if delta_w < tol:
    print(f"第{iter_idx}次迭代收敛")
    break
w = w_new # 更新参数, 进入下一次迭代

return w

# 训练逻辑回归模型
w = irls_logistic(X, y)
print("训练得到的逻辑回归参数: ", w)

# 绘制原始数据
plt.figure(figsize=(8, 6))
plt.scatter(X[y==0, 1], X[y==0, 2], color="red", label="Not Admitted") # 未录取
plt.scatter(X[y==1, 1], X[y==1, 2], color="blue", label="Admitted") # 录取

# 绘制决策边界:  $w_0 + w_1x_1 + w_2x_2 = 0$ ,  $x_2 = (-w_0 - w_1x_1)/w_2$ 
x1_min, x1_max = X[:, 1].min() - 1, X[:, 1].max() + 1
x1_vals = np.linspace(x1_min, x1_max, 100)
x2_vals = (-w[0] - w[1] * x1_vals) / w[2]

plt.plot(x1_vals, x2_vals, color="black", linewidth=2, label="Decision Boundary")

plt.xlabel("Exam 1 Score")
plt.ylabel("Exam 2 Score")
plt.title("Logistic Regression Classification Result (IRLS)")
plt.legend()
plt.grid(True)
plt.show()

# 预测与准确率计算
y_pred = (sigmoid(X @ w) > 0.5).astype(int) # 概率>0.5预测为1, 否则为0
accuracy = np.mean(y_pred == y)
print(f"分类准确率: {accuracy:.2%}")

```